🎯 Spark SQL, DataFrames i optymalizacja zapytań

Zaawansowane przetwarzanie danych — bo SQL nigdy nie umiera 💀➡️🚀

Python w Zastosowaniach Big Data | Wykład 3

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Szybkie przypomnienie — RDD i DataFrame
2. Spark SQL i DataFrame API w akcji
3. Agregacje, joiny i funkcje okienkowe (Window Functions)
4. UDF-y — własne funkcje w Spark
5. Catalyst Optimizer — jak Spark myśli 🧠
6. Performance tips — żeby nie czekać do jutra ⚡

> 🎯 **Po tym wykładzie:** Będziesz pisać zapytania Spark SQL jak pro i rozumieć, dlaczego działają szybko (lub wolno)!

💡 Dlaczego to ważne?

Czy wiesz, że **Uber przetwarza ponad 100 petabajtów danych** w Apache Spark?

A większość tych zapytań to... **zwykły SQL!** 🤯

Firma	Dane dziennie	Główne narzędzie
Uber	100+ PB	Spark SQL
Netflix	500 mld zdarzeń	Spark + Kafka
Allegro	Miliardy kliknięć	Spark + BigQuery

> SQL to **lingua franca** świata danych. Spark SQL pozwala używać go na **petabajtach**!

🔄 Szybkie przypomnienia — co już wiemy

Z poprzednich wykładów:

- **RDD** (Resilient Distributed Dataset) — niskopoziomowa abstrakcja Spark
- **DataFrame** — ustrukturyzowane dane z kolumnami i typami (jak tabela SQL)
- **SparkSession** — punkt wejścia do Spark 2.0+
- **Transformacje** są leniwe (lazy), **akcje** uruchamiają obliczenia

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Wyklad3") \
    .master("local[*]") \
    .getOrCreate()
```

🤔 RDD vs DataFrame — kiedy co?

Cecha	RDD	DataFrame
Typ danych	Dowolny obiekt Python	Wiersze + kolumny (schema)
Optymalizacja	❌ Brak	✅ Catalyst Optimizer
Wydajność	Wolniejsze	**10–100× szybsze**
Typowanie	Ręczne	Schema + typy
SQL support	❌	✅

> 💡 **Zasada kciuka:** Używaj DataFrame / Spark SQL **zawsze**, chyba że musisz operować na surowych obiektach.

🔑 Co to jest Spark SQL?

Spark SQL = moduł Spark do pracy z danymi ustrukturyzowanymi

Trzy sposoby na to samo:

1. **DataFrame API** — programistycznie (Python/Scala)
2. **SQL queries** — klasyczny SQL na DataFrames
3. **Mix obu** — bo czemu nie? 😎

```
# Sposób 1: DataFrame API
df.select("name", "age").filter(df.age > 21)

# Sposób 2: SQL
spark.sql("SELECT name, age FROM people WHERE age > 21")
```

> Oba generują **ten sam plan wykonania** — Catalyst je optymalizuje identycznie!

📊 Ładujemy dane — punkt startowy

Na dziś pracujemy z danymi o zamówieniach e-commerce:

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F  # 👈 Zawsze importujemy jako F

spark = SparkSession.builder.appName("SparkSQL").getOrCreate()

orders = spark.read.csv(
    "orders.csv",
    header=True,
    inferSchema=True  # 👈 Spark sam wykryje typy
)

orders.printSchema()
orders.show(5)
```

> 🎯 `functions as F` to **konwencja** w świecie PySpark — zapamiętajcie!

🔧 DataFrame API — select i filter

Podstawowe operacje na kolumnach:

```
from pyspark.sql.functions import col, upper

# select – wybierz kolumny
orders.select("order_id", "customer_name", "total")

# select z transformacją
orders.select(
    col("order_id"),
    upper(col("customer_name")).alias("name_upper"),  # 👈 alias = AS w SQL
    col("total") * 1.23                                # 👈 arytmetyka
)

# filter / where – to samo, dwie nazwy
orders.filter(col("total") > 100)
orders.where(col("status") == "completed")
```

🔧 DataFrame API — withColumn i when

Dodawanie i modyfikacja kolumn:

```
from pyspark.sql.functions import col, when, lit

# withColumn – dodaj/zmień kolumnę
orders_with_vat = orders.withColumn(
    "total_with_vat",
    col("total") * 1.23  # 👈 Polski VAT 23%
)

# when/otherwise – odpowiednik CASE WHEN w SQL
orders_categorized = orders.withColumn(
    "size",
    when(col("total") < 50, lit("small"))
    .when(col("total") < 200, lit("medium"))
    .otherwise(lit("large"))  # 👈 Zawsze dodawaj otherwise!
)
```

> ⚠️ `withColumn` **nie zmienia** oryginalnego DataFrame — tworzy nowy! (immutability)

🤔 Think About It

Pytanie: Co się stanie, jeśli zapomnimy `otherwise()` w `when()`?

```
df.withColumn(
    "category",
    when(col("price") > 100, "expensive")
    .when(col("price") > 50, "medium")
    # brak .otherwise(...) 🤔
)
```

A) Błąd kompilacji 💥

B) Wartości `null` dla niepasujących wierszy

C) Spark zgadnie wartość domyślną

📊 Agregacje — groupBy i agg

```
from pyspark.sql.functions import (
    count, sum as _sum, avg, min as _min, max as _max
)

# Proste grupowanie
orders.groupBy("status").count().show()

# Wiele agregacji naraz
orders.groupBy("category").agg(
    count("order_id").alias("num_orders"),
    _sum("total").alias("revenue"),       # 👈 sum jest shadowed w Pythonie!
    avg("total").alias("avg_order"),
    _max("total").alias("max_order")
).orderBy(F.desc("revenue")).show()
```

> ⚠️ Importujemy `sum as _sum`, bo `sum` to wbudowana funkcja Pythona!

📊 Agregacje — pivot

Pivot = obrót danych z wierszy na kolumny (jak tabela przestawna w Excelu):

```
# Przychody per kategoria per miesiąc
monthly_revenue = (
    orders
    .groupBy("category")
    .pivot("month", ["Jan", "Feb", "Mar"])  # 👈 określ wartości!
    .agg(_sum("total"))
)

monthly_revenue.show()
# +----------+------+------+------+
# |  category|   Jan|   Feb|   Mar|
# +----------+------+------+------+
# |Electronics| 5000|  7200|  6100|
# |   Clothing| 3200|  2800|  4500|
# +----------+------+------+------+
```

> 💡 **Podawaj wartości pivotu** jawnie — inaczej Spark musi najpierw skanować dane.

🔗 Joiny — łączenie DataFrames

Spark obsługuje **wszystkie** typy joinów:

```
customers = spark.read.csv("customers.csv", header=True, inferSchema=True)

# Inner join – tylko pasujące wiersze
result = orders.join(customers, "customer_id")  # 👈 klucz po nazwie

# Left join – wszystko z lewej tabeli
result = orders.join(customers, "customer_id", "left")

# Full outer join
result = orders.join(
    customers,
    orders.customer_id == customers.customer_id,  # 👈 warunek jawny
    "full_outer"
)
```

🔗 Joiny — pełna mapa

Typ joina	Co dostajemy	Kiedy używać
`inner`	Tylko pasujące	Domyślny, najczęstszy
`left` / `left_outer`	Wszystko z lewej + pasujące z prawej	Chcemy zachować „główną" tabelę
`right` / `right_outer`	Wszystko z prawej + pasujące z lewej	Rzadko — zamień strony i użyj left
`full_outer`	Wszystko z obu	Pełny obraz, dużo null-i
`cross`	Iloczyn kartezjański	Ostrożnie! n × m wierszy 💥
`left_semi`	Wiersze z lewej, które mają match	Jak `WHERE EXISTS`
`left_anti`	Wiersze z lewej, które **nie** mają matcha	Jak `WHERE NOT EXISTS`

> 🎯 `left_semi` i `left_anti` to **ukryte perełki** — zapamiętajcie je!

🔗 Semi i Anti Join — praktyka

```
# Klienci, którzy złożyli co najmniej jedno zamówienie
active_customers = customers.join(
    orders, "customer_id", "left_semi"  # 👈 WHERE EXISTS
)

# Klienci, którzy NIGDY nie zamówili
inactive_customers = customers.join(
    orders, "customer_id", "left_anti"  # 👈 WHERE NOT EXISTS
)
```

> 💡 Semi join zwraca **tylko kolumny z lewej tabeli** — w przeciwieństwie do inner join, który zwraca obie!

🪟 Window Functions — supermoc SQL-a

Funkcje okienkowe = obliczenia na grupie wierszy **bez** redukcji wyniku

Wyobraź sobie: chcesz policzyć **ranking** sprzedawców w każdej kategorii. `groupBy` dałby Ci jedną liczbę per kategoria, ale Ty chcesz **ranking każdego wiersza**.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, rank, dense_rank

# Definiujemy okno
window_spec = Window \
    .partitionBy("category") \
    .orderBy(F.desc("total"))  # 👈 od największego
```

> 🎯 **Window = partitionBy + orderBy**. Zapamiętaj to jak mantrę!

Window Functions — ranking

```
from pyspark.sql.functions import row_number, rank, dense_rank

window_spec = Window.partitionBy("category").orderBy(F.desc("total"))

ranked = orders.select(
    "category", "order_id", "total",
    row_number().over(window_spec).alias("row_num"),   # 1, 2, 3, 4...
    rank().over(window_spec).alias("rank"),             # 1, 2, 2, 4...
    dense_rank().over(window_spec).alias("dense_rank")  # 1, 2, 2, 3...
)

ranked.show()
```

Funkcja	Remisy	Luki
`row_number`	Brak — każdy ma unikalny numer	—
`rank`	Ten sam numer	✅ Luka po remisie
`dense_rank`	Ten sam numer	❌ Bez luk

🪟 Window Functions — lag, lead i sumy bieżące

```
from pyspark.sql.functions import lag, lead, sum as _sum

time_window = Window.partitionBy("customer_id").orderBy("order_date")

analysis = orders.select(
    "customer_id", "order_date", "total",
    lag("total", 1).over(time_window).alias("prev_order"),       # 👈 poprzednie
    lead("total", 1).over(time_window).alias("next_order"),      # 👈 następne
    _sum("total").over(
        time_window.rowsBetween(
            Window.unboundedPreceding, Window.currentRow          # 👈 running total
        )
    ).alias("cumulative_total")
)
```

> 💡 `lag` = "cofnij się", `lead` = "podejrzyj następny". Jak lustro wsteczne i GPS! 🚗

💬 SQL na DataFrames — createOrReplaceTempView

Wolisz pisać SQL? Spark to umożliwia!

```
# Rejestrujemy DataFrame jako „tabelę" w Spark SQL
orders.createOrReplaceTempView("orders")  # 👈 tymczasowy widok
customers.createOrReplaceTempView("customers")

# Teraz możemy pisać SQL jak w bazie danych!
result = spark.sql("""
    SELECT c.customer_name, COUNT(*) AS num_orders, SUM(o.total) AS revenue
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.status = 'completed'
    GROUP BY c.customer_name
    HAVING revenue > 1000
    ORDER BY revenue DESC
""")

result.show()
```

💬 SQL + DataFrame API — najlepsze z obu światów

```
# Zaczynamy od SQL...
top_customers = spark.sql("""
    SELECT customer_id, SUM(total) AS revenue
    FROM orders
    GROUP BY customer_id
    HAVING revenue > 5000
""")

# ...i kontynuujemy DataFrame API!
final = (
    top_customers
    .join(customers, "customer_id")
    .select("customer_name", "revenue")
    .withColumn("tier",
        when(col("revenue") > 10000, "Gold")
        .when(col("revenue") > 7000, "Silver")
        .otherwise("Bronze")
    )
    .orderBy(F.desc("revenue"))
)
```

> 🔑 **Mix SQL z DataFrame API** to najczęstszy wzorzec w produkcji!

🤔 Think About It

Pytanie: Który sposób jest **szybszy** — DataFrame API czy spark.sql()?

```
# Opcja A: DataFrame API
df.filter(col("age") > 21).select("name").show()

# Opcja B: SQL
spark.sql("SELECT name FROM people WHERE age > 21").show()
```

Odpowiedź: Żaden — oba generują **identyczny plan wykonania**! 🤯

Catalyst Optimizer optymalizuje oba identycznie. Wybierz ten, który jest bardziej czytelny.

🛠️ UDF — User Defined Functions

Kiedy wbudowane funkcje Spark **nie wystarczają**, tworzymy własne:

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# Sposób 1: Klasyczny UDF
def categorize_price(price: float) -> str:
    if price > 500:
        return "premium"
    elif price > 100:
        return "standard"
    return "budget"

categorize_udf = udf(categorize_price, StringType())  # 👈 rejestracja

orders.withColumn("price_tier", categorize_udf(col("total"))).show()
```

🛠️ UDF — dekorator @udf (czystszy sposób)

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())  # 👈 dekorator — wygodniej!
def clean_name(name: str) -> str:
    if name is None:
        return "Unknown"
    return name.strip().title()

# Rejestracja UDF dla SQL
spark.udf.register("clean_name_sql", clean_name)

# Teraz działa w DataFrame API i SQL!
orders.withColumn("clean_name", clean_name(col("customer_name")))
spark.sql("SELECT clean_name_sql(customer_name) FROM orders")
```

⚠️ UDF — pułapki wydajnościowe

UDF-y w Pythonie są WOLNE! Dlaczego?

```
                    Szybka ścieżka (wbudowane funkcje)
DataFrame ──▶ [Catalyst] ──▶ [Tungsten / JVM] ──▶ Wynik    ✅ Szybko!

                    Wolna ścieżka (Python UDF)
DataFrame ──▶ [JVM → Python → JVM] ──▶ Wynik               ❌ Serializacja!
     per każdy wiersz ↑ to boli 😭
```

Podejście	Wydajność	Kiedy używać
Wbudowane `F.` funkcje	⚡⚡⚡	**Zawsze** gdy się da
Pandas UDF (vectorized)	⚡⚡	Operacje na kolumnach, NumPy/pandas
Python UDF	⚡	Ostateczność — skomplikowana logika

> 🎯 **Zasada #1:** Sprawdź, czy `pyspark.sql.functions` nie ma już tego, czego potrzebujesz!

🧠 Catalyst Optimizer — jak Spark myśli

Catalyst to silnik optymalizacji zapytań w Spark SQL.

Każde zapytanie przechodzi przez **4 fazy**:

```
1. Parsed Logical Plan     — co napisał programista
         ↓
2. Analyzed Logical Plan   — sprawdzone kolumny, typy, tabele
         ↓
3. Optimized Logical Plan  — optymalizacje (predicate pushdown, column pruning...)
         ↓
4. Physical Plan           — konkretna strategia wykonania
```

> 💡 Dlatego DataFrame API i SQL dają **ten sam wynik** — oba przechodzą przez Catalyst!

🧠 Catalyst — predicate pushdown

Predicate pushdown = Spark przesuwa filtry **jak najwcześniej**, żeby czytać mniej danych:

```
# Napisaliśmy:
orders.join(customers, "customer_id").filter(col("status") == "completed")

# Catalyst optymalizuje do:
orders.filter(col("status") == "completed").join(customers, "customer_id")
# 👈 Filter PRZED joinem — mniej danych do joinowania!
```

Inne optymalizacje Catalyst:

- **Column pruning** — czyta tylko potrzebne kolumny
- **Constant folding** — oblicza stałe wyrażenia z góry
- **Join reordering** — zmienia kolejność joinów na optymalną

🔍 explain() — podglądamy plan zapytania

```
result = (
    orders
    .filter(col("status") == "completed")
    .groupBy("category")
    .agg(_sum("total").alias("revenue"))
    .orderBy(F.desc("revenue"))
)

result.explain(True)  # 👈 True = pokaż wszystkie fazy
```

```
== Physical Plan ==
*(2) Sort [revenue DESC]
+- Exchange rangepartitioning(revenue DESC)  <-- shuffle!
   +- *(1) HashAggregate [category], [sum(total)]
      +- *(1) Filter (status = completed)
         +- FileScan csv [category, total, status]  <-- column pruning!
```

> 🎯 **Używaj** `explain()` — to Twój najlepszy przyjaciel przy debugowaniu wydajności!

⚡ Performance Tip #1 — Caching

Wielokrotnie używasz tego samego DataFrame? **Cache it!**

```
# Cachowanie w pamięci (domyślnie MEMORY_AND_DISK)
frequent_df = orders.filter(col("status") == "completed")
frequent_df.cache()  # 👈 leniwe — cache dopiero przy pierwszej akcji
frequent_df.count()  # 👈 teraz dane są w pamięci

# Później — zwolnij pamięć
frequent_df.unpersist()
```

Poziom	Opis
`MEMORY_ONLY`	Tylko RAM — najszybciej, ryzyko OOM
`MEMORY_AND_DISK`	RAM + dysk — domyślne, bezpieczne
`DISK_ONLY`	Tylko dysk — gdy mało RAM

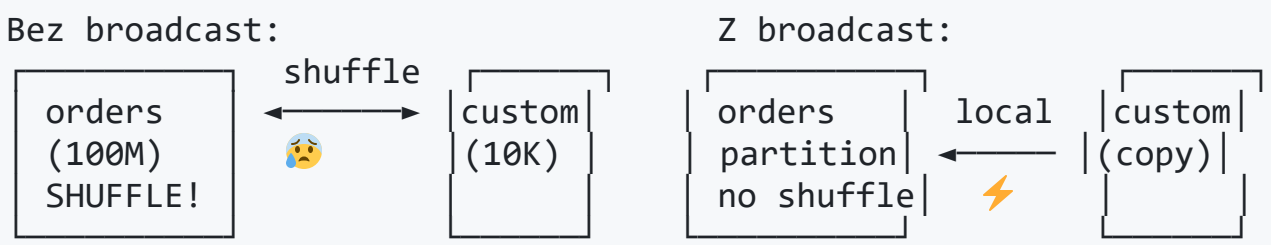
> ⚠️ Cachuj **tylko** gdy DataFrame jest używany **wielokrotnie**!

⚡ Performance Tip #2 — Broadcast Join

Mały DataFrame + duży DataFrame = **broadcast join**!

```python
from pyspark.sql.functions import broadcast

# Spark automatycznie broadcastuje df < 10MB
# Ale możemy wymusić:
result = orders.join(
    broadcast(customers),  # 👈 wyślij customers do KAŻDEGO executora
    "customer_id"
)
```



> 🎯 Broadcast = mała tabela jest **kopiowana** do każdego executora. Zero shuffli!

⚡ Performance Tip #3 — Unikaj shuffli

Shuffle = przenoszenie danych między executorami. To **najdroższa** operacja w Spark!

Operacje powodujące shuffle:

- `groupBy()` + agregacja
- `join()` (poza broadcast)
- `orderBy()` / `sort()`
- `repartition()`
- `distinct()`

```
# ❌ Dwa shuffle — groupBy + orderBy
orders.groupBy("category").count().orderBy("count")

# ✅ Lepiej — ogranicz dane PRZED grupowaniem
orders.filter(col("year") == 2026) \
      .groupBy("category").count() \
      .orderBy("count")
```

> 💡 **Filtruj wcześnie, shuffluj późno** — złota zasada Spark!

⚡ Performance Tip #4 — Partycjonowanie i AQE

```
# Sprawdź liczbę partycji
print(orders.rdd.getNumPartitions())  # np. 200 (domyślne)

# Zmniejsz partycje (bez shuffla!)
orders_coalesced = orders.coalesce(10)  # 👈 zmniejsza BEZ shuffla

# Zmień liczbę partycji (z shufflem)
orders_repartitioned = orders.repartition(50)

# Partycjonowanie po kolumnie — świetne do joinów!
orders.repartition("customer_id")  # 👈 ten sam klucz = ta sama partycja
```

AQE (Adaptive Query Execution) — Spark 3.0+:

- Automatycznie dostosowuje plan **w trakcie wykonania**
- Łączy małe partycje po shuffle
- Domyślnie **włączone** od Spark 3.2 ✅

🤔 Think About It

Pytanie: Masz tabelę `orders` (500M wierszy) i `categories` (50 wierszy). Jak je połączyć najwydajniej?

A) `orders.join(categories, "category_id")`
B) `orders.join(broadcast(categories), "category_id")`
C) `categories.join(orders, "category_id")`

💻 Live Demo — kompletny przykład

Analiza zamówień e-commerce — łączymy **wszystkie** dzisiejsze techniki:

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder.appName("Demo").getOrCreate()

# Dane testowe
orders_data = [
    (1, "Alice", "Electronics", 1200, "2026-01-15"),
    (2, "Bob", "Clothing", 85, "2026-01-16"),
    (3, "Alice", "Electronics", 450, "2026-02-01"),
    (4, "Charlie", "Books", 35, "2026-02-10"),
    (5, "Bob", "Electronics", 2100, "2026-02-15"),
    (6, "Alice", "Clothing", 120, "2026-03-01"),
    (7, "Charlie", "Clothing", 95, "2026-03-05"),
    (8, "Bob", "Books", 45, "2026-03-10"),
]

schema = ["order_id", "customer", "category", "total", "date"]
orders = spark.createDataFrame(orders_data, schema)
```

💻 Demo — agregacje i rankingi

```
# Przychód per kategoria
revenue = (
    orders
    .groupBy("category")
    .agg(
        F.count("order_id").alias("num_orders"),
        F.sum("total").alias("revenue"),
        F.round(F.avg("total"), 2).alias("avg_order")
    )
    .orderBy(F.desc("revenue"))
)

revenue.show()
# +----------+----------+-------+---------+
# |  category|num_orders|revenue|avg_order|
# +----------+----------+-------+---------+
# |Electronics|         3|   3750|  1250.00|
# |  Clothing|         3|    300|   100.00|
# |     Books|         2|     80|    40.00|
# +----------+----------+-------+---------+
```

💻 Demo — Window Functions w akcji

```
# Ranking klientów w każdej kategorii
window = Window.partitionBy("category").orderBy(F.desc("total"))

ranked = orders.select(
    "customer", "category", "total",
    F.rank().over(window).alias("rank"),
    F.sum("total").over(
        Window.partitionBy("customer")
        .orderBy("date")
        .rowsBetween(Window.unboundedPreceding, Window.currentRow)
    ).alias("running_total")
)

ranked.show()
# +--------+----------+-----+----+-------------+
# |customer|  category|total|rank|running_total|
# +--------+----------+-----+----+-------------+
# |     Bob|Electronics| 2100|   1|         2230|
# |   Alice|Electronics| 1200|   2|         1770|
# |   Alice|  Clothing|  120|   1|          1770|
# +--------+----------+-----+----+-------------+
```

💻 Demo — SQL + DataFrame mix

```
# Rejestrujemy widok
orders.createOrReplaceTempView("orders")

# SQL dla złożonej logiki
vip_customers = spark.sql("""
    SELECT customer, SUM(total) AS total_spent, COUNT(*) AS orders_count
    FROM orders
    GROUP BY customer
    HAVING total_spent > 200
""")

# DataFrame API dla finalnej transformacji
result = (
    vip_customers
    .withColumn("tier",
        F.when(F.col("total_spent") > 2000, F.lit("🥇 Gold"))
        .when(F.col("total_spent") > 500, F.lit("🥈 Silver"))
        .otherwise(F.lit("🥉 Bronze"))
    )
    .orderBy(F.desc("total_spent"))
)

result.show(truncate=False)
```

💻 Demo — sprawdźmy plan wykonania

```
result.explain(True)
```

```
== Optimized Logical Plan ==
Sort [total_spent DESC]
+- Project [customer, total_spent, orders_count, CASE ... AS tier]
   +- Filter (total_spent > 200)          ← 👈 HAVING wepchnięty w plan
      +- Aggregate [customer]
         +- Relation [orders]

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true       ← 👈 AQE w akcji!
+- Sort [total_spent DESC]
   +- Exchange rangepartitioning          ← 👈 shuffle na sortowanie
      +- HashAggregate [customer]
         +- LocalTableScan [...]
```

> Catalyst zepchnął HAVING jako filtr i AQE dostosował partycje!

⚠️ Częste pułapki

#	Pułapka	Dlaczego boli	Rozwiązanie
1	Python UDF na dużych danych	Serializacja JVM↔️Python	Użyj wbudowanych `F.` funkcji
2	`collect()` na dużym DataFrame	Ściąga WSZYSTKO do drivera	Użyj `show()`, `take()`, `head()`
3	Brak `otherwise()` w `when()`	Ciche null-e	Zawsze dodaj `otherwise()`
4	Cross join „przez przypadek"	n × m wierszy — OOM	Sprawdź warunki joinów
5	Cache bez `unpersist()`	Wyciek pamięci	Zwolnij cache gdy niepotrzebny
6	`repartition()` zamiast `coalesce()`	Niepotrzebny shuffle	`coalesce()` do zmniejszania

✅ Best Practices — ściągawka

1. 🔧 **Import:** `from pyspark.sql import functions as F` — zawsze
2. ⚡ **Filtruj wcześnie** — mniej danych = szybsze operacje
3. 📡 **Broadcast** małe tabele przy joinach
4. 🧠 `explain()` — sprawdzaj plan przed uruchomieniem na produkcji
5. 💾 **Cache** tylko wielokrotnie używane DataFrames
6. 🐍 **Unikaj Python UDF** — Pandas UDF lub wbudowane funkcje
7. 📁 **Parquet > CSV** — kolumnowy format = column pruning

```
# ✅ Wzorzec produkcyjny
result = (
    spark.read.parquet("orders/")          # Parquet, nie CSV
    .filter(F.col("year") == 2026)         # Filtr wcześnie
    .join(broadcast(categories), "cat_id") # Broadcast
    .groupBy("category")
    .agg(F.sum("total").alias("revenue"))
)
result.explain()                           # Sprawdź plan!
```

📝 Podsumowanie — kluczowe wnioski

1. ✅ **Spark SQL = DataFrame API = SQL** — ten sam silnik, różna składnia
2. ✅ **Window Functions** to supermoc — ranking, sumy bieżące, lag/lead
3. ✅ **Catalyst Optimizer** automatycznie optymalizuje zapytania
4. ✅ **Unikaj shuffli** — broadcast, filtruj wcześnie, coalesce
5. ✅ `explain()` jest Twoim najlepszym narzędziem diagnostycznym

```
Twój workflow:
Napisz zapytanie → explain() → zoptymalizuj → profit 🎉
```

🔮 Co dalej?

Następny wykład (Wykład 4): Spark Structured Streaming — przetwarzanie danych w czasie rzeczywistym 🌊

Do przeczytania:

- Spark SQL Guide
- Rozdziały 4–7 z „Learning Spark, 2nd Edition" (O'Reilly)

Ćwiczenie:

- Ćwiczenie 3 — Spark SQL na danych e-commerce w Google Colab 📓

❓ Pytania?

Dziękuję za uwagę! 🎉

Zanim pójdziecie — **jedno pytanie na rozgrzewkę do ćwiczeń:**

> Macie tabelę `products` (1000 wierszy) i `sales` (50 milionów wierszy).
> Jak je połączyć najwydajniej?

💬 Odpowiedzi na czacie lub na żywo!